

SC2002 – OBJECT-ORIENTED DESIGN AND PROGRAMMING

Project Title: Internship Management System

Report of Project Structure Design & Functionality

AY2025/26 Sem 1 | SCEC, Group 4

Declaration of Original Work for SC2002 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course/ Year	Lab Group	Signature/ Date
Jolie Loke Zhi Xuan	Computer Engineering/ Year 2	SCEC, Group 4	 18/11/2025
Natarajan Advait	Computer Engineering/ Year 2	SCEC, Group 4	 18/11/2025
Sahana Anandhan	Computer Engineering/ Year 2	SCEC, Group 4	 18/11/2025
Sivakumar Yasuvanthi	Computer Engineering/ Year 2	SCEC, Group 4	 18/12/2025

1. Github Link: <https://github.com/advnat22/SC2002>

2. Design Considerations:

2.1. Approach Taken

The Internship Management System is a Java Command Line Interface (CLI) application that allows students, company representatives, and career center staff to manage internship postings, applications, and approvals efficiently. The system employs loosely coupled classes organized primarily around the core roles of the application, leveraging object-oriented principles such as abstraction, inheritance, and polymorphism for clarity and modularity, where each major entity (User types, Internship) is modeled as a class. The design separates responsibilities among different user roles (Student, CompanyRepresentative, CareerCenterStaff) using inheritance and polymorphism. Data persistence relies on CSV file storage with each class responsible for loading and saving its own data. The console-based UI is simple but sufficiently modular.

In summary, our approach balances simplicity required for an internship management application with sound OOP design principles and patterns, providing a stable foundation that meets project requirements and supports future growth. We have implemented the application with an ECB Pattern. This architecture ensures dependencies between classes is minimized and makes the entire system flexible, extensible, and simple to maintain. New features can be added with minimal changes to existing code, letting the system evolve smoothly as project requirements grow.

2.2. Assumptions Made

- The system supports a single user at a time interacting via console menus, with data stored in predefined CSV files and minimal validation.
- The number of applications or internships is limited by constants (e.g., maximum applications allowed per student).
- The system is designed for small to medium scale, prioritizing clarity over performance.
- New students and Career Center Staff cannot be created and can only be done by editing their respective csv file.

2.3. Design Patterns Used

2.3.1. Entity- Control- Boundary (ECB) Pattern:

The design follows the Entity-Controller-Boundary (ECB) architectural pattern

- Entity classes represent the core data models, such as Student, Internship, CompanyRepresentative, and CareerCenterStaff. These classes encapsulate the essential attributes and behaviors, including CSV data loading and saving operations.
- Controller classes handle user inputs and coordinate the application's response for specific user roles. In our internship management system, classes like StudentController and CompanyRepController manage role-specific logic and workflows. The main application

class, `InternshipManagementApp`, oversees overall flow and delegates tasks to these controllers, ensuring clear separation of responsibilities and modular design.

- Boundary classes are implemented via user-specific console menus embedded within each user type class. These boundary classes serve as the system's interface, interacting with users through text-based menus and facilitating commands that invoke entity or controller logic.

2.3.2. Component-Based Menus:

Control-based menus (also called component-based menus) are a design approach where menu options and user interface flows are encapsulated within distinct components or classes. This modularizes the UI, making each menu or sub-menu a reusable, extendable unit that controls its user interaction independently. In our internship management system, the `CompanyRepresentative` class's `displayMenu()` method does not directly handle the menu's UI. Instead, it calls a dedicated `CompanyRepMenuDisplay` class responsible for rendering and managing the company representative's menu options and flows. This separation maintains modularity by isolating menu UI logic in reusable components while keeping the user classes focused on role-specific behavior and control. This design separates UI interaction logic according to user roles, keeping menus focused and relevant to each user type and aids in extensibility, allowing new user roles with customized menus to be added easily without changing unrelated code. Also, maintenance is simplified because UI changes or new options are confined to the specific user class responsible for that menu.

2.4. Coupling and Cohesion:

Coupling refers to the degree of interdependence between software modules or classes. Low coupling means that modules communicate with each other through well-defined interfaces and minimize knowledge of each other's internal details. This reduces the ripple effect when changes are made, enhancing modularity and easing maintenance. Our design exhibits low coupling by separating concerns into entity classes, a central controller, and boundary/UI classes. Each module interacts through clean, minimal interfaces. For instance, entities handle their own data persistence, while the controller manages flow without directly accessing entity internals.

Cohesion measures how closely related and focused the responsibilities of a single module or class are. High cohesion means a class or module has a clear, single purpose, improving readability, making it easier to manage and less prone to bugs. Our classes show high cohesion by focusing on a single responsibility. For example, the `Student` class manages student-specific behaviors and data but does not involve UI handling or controller logic. Similarly, the `Internship` class concentrates on internship attributes and related logic.

2.5. Usage of OOPS Principles:

2.5.1. Abstraction:

Abstraction is where a programmer hides all but the relevant data about an object in order to reduce complexity and efficiency. The abstract class `User` encapsulates shared properties and behavior for all user

types. Abstract methods (e.g., `displayMenu()`) enforce user-specific implementations in subclasses. The abstract class `User` defines a general interface and hides irrelevant implementation details from its subclasses (`Student`, `CompanyRepresentative`, `CareerCenterStaff`). Each subclass only needs to know about the methods it must implement, such as `displayMenu()`, abstracting away the details of authentication and common attributes like `userID`, `name`, and `password`.

2.5.2. Encapsulation/ Information Hiding:

Encapsulation is used to build a barrier around an object's data, preventing the unauthorized access of data. Important data fields within each class are declared private or protected to hide complexity and prevent unauthorized access. For example, private fields with public methods control access to sensitive data like passwords and lists of internships or applications. Internal implementation details are hidden within classes. Direct access to the attributes are restricted and are retrieved by getters and setters. For example, in our `Student` class, the list of applied internships is private, and can only be accessed or modified via public methods like `getAppliedInternships()` or methods that allow applying for internships.

Information Hiding hides the details and implementation of the class from users. With information hiding, the `Controller` class can simply call these methods without understanding the implementation for those methods. The same goes for a `Boundary` class that calls methods from a `Controller` class.

2.5.3. Inheritance

Inheritance allows us to derive new classes from existing classes. The child class inherits data attributes and methods from the parent class. This "is-a" relationship between the child class and parent class can be visualized in our class diagram. Inheritance is helpful as it reduces developer effort and code length, as it enables the reuse of code when implementing new classes. For example, in the model component, the `Student` and `Staff` classes are inheriting methods from the `User` class. This allows common methods such as `getID()` and to be inherited and reused.

2.5.4. Polymorphism:

Polymorphism allows objects of different types to be treated as objects of a common type. Polymorphism allows an object to have different implementations of a method in a subclass that it inherited from its parent. Our `User` abstract class defines the `displayMenu()` method, which is overridden by each subclass (`Student`, `CompanyRepresentative`, and `CareerCenterStaff`) to implement role-specific menu displays and options. Polymorphism is also used to allow objects to take on multiple forms and behaviors. For example, the different subclasses of the `User` class are able to override methods such as the `updatePassword()` and `login()`.

2.6. Design Principles (SOLID)

2.6.1. Single Responsibility Principle (SRP):

The Single Responsibility Principle (SRP) recommends that each class should have a clear and singular responsibility, avoiding unrelated tasks. By adhering to the SRP, we can minimize the ripple effect of changes, simplifying the process of modifying, testing, and reusing code, resulting in more maintainable and robust software design. In essence, this would achieve the low coupling effect, approaching one of the

good design principles.

Each class in our system has one primary responsibility or focus area. For example, the

- Student class manages student-specific data and behaviors, like storing applications and managing internship statuses, but it does not handle unrelated concerns such as file I/O beyond loading and saving its own data.
- The Internship class is solely responsible for internship-related attributes and logic such as slots, status, and visibility.
- The CompanyRepresentative and CareerCenterStaff classes manage their own user roles and related functions.
- The InternshipManagementApp class acts as a central coordinator/controller managing program flow, user sessions, and coordinating interactions, but it does not directly manipulate lower-level user or internship data details.
- User interface concerns are encapsulated within respective user menu display methods inside each user class, separating UI concerns from data and control logic.

This separation means each class has a clear purpose, making the codebase easier to maintain, test, and extend. Changes in one responsibility area are less likely to impact unrelated parts of the system, adhering well to the SRP.

2.6.2. Open/Closed Principle (OCP):

The Open/Closed Principle (OCP) states that classes should be open for extension but closed for modification, allowing for the addition of new functionality without changing existing code. OCP can be implemented through abstraction, inheritance, and polymorphism. This means that a certain class allows for more inheritance from it (if necessary), but the methods of the class are kept consistent so that existing classes that realise from it would not be disrupted.

One example we have in our code is in Company Representative and Student inheriting from User. The User class provides basic attributes that all Users in the system should have like userID, while the CompanyRepresentative and Student class implements specific attributes that are used only by them like internships for Company Representative, which stores the internships created them, and appliedInternships for Student, which stores the internships a student has registered themselves into. By creating a new class which inherits from the parent User class, we could create new user classes in the future without changing any code inside User, Student, or CompanyRepresentative. This ensures the extensibility of our code.

2.6.3. Liskov Substitution Principle (LSP):

Barbara Liskov stated that “a user of a base class should continue to function properly if a derivative of that base class is passed to it”. This means that even if arguments were to pass to the subclass instead of the base class, the pre-conditions and postconditions of the subclass method are met. Essentially, it means that the derived class is substitutable for its base class.

We have applied the Liskov Substitution Principle (LSP) in our internship management system through the

use of inheritance and polymorphism with our User abstract class and its subclasses. In our system, instances of subclasses (Student, CompanyRepresentative, CareerCenterStaff) can be used wherever the User type is expected. For example, the controller or system treats users uniformly as User objects, invoking common methods like `displayMenu()` without needing to know the specific subclass. All subclasses adhere to the expected behavior defined by the abstract User class, ensuring substitutability. This design allows seamless extension by adding new user types without impacting existing code, fulfilling LSP.

2.6.4. Interface Segregation Principle (ISP):

Interface segregation states that when using interfaces, each interface should be as small as possible. Many specific interfaces are generally better than one general interface and a class should implement many interfaces as opposed to one large “fat” interface.

In our internship management system, while we do not explicitly define multiple separate interfaces, the design inherently reflects the ISP by ensuring that each class only implements methods relevant to its role and responsibilities. For example, the User abstract class defines only essential methods like `displayMenu()`, and each subclass (Student, CompanyRepresentative, CareerCenterStaff) overrides these methods to provide behavior specific to that role. This approach keeps our code modular and flexible, as user roles are not burdened with irrelevant method implementations. It also facilitates ease of extending functionality for new user types, adhering to the ISP principle in spirit through focused class responsibilities rather than through numerous explicit interfaces.

2.6.5. Dependency Injection Principle (DIP):

The dependency injection principle suggests that higher modules must not depend on lower modules, but both should depend on abstraction. In other words, instead of directly depending on the concrete class to perform some operation, we can depend on the interfaces, which are less likely to be changed.

Dependency Inversion Principle (DIP) is applied through the use of abstraction by depending on the abstract User class rather than directly on concrete classes like Student or CompanyRepresentative. The controller or other high-level logic interacts with users through references to the User abstract class, relying on the abstraction rather than concrete implementations. This allows the system to remain flexible and extensible. When new user types are added (new subclasses of User), this can be done without modifying the existing code that depends on the User abstraction, thus avoiding tight coupling to specific user implementations. This use of abstraction in our design approximates the DIP by inverting the dependency away from concrete details toward stable abstractions. Although we do not use interfaces or formal dependency injection frameworks, this design still improves maintainability and extensibility.

2.7. Extensibility and Maintainability

Extensibility is enabled by leveraging inheritance, polymorphism, and adherence to design principles like the OCP. Our abstract User class allows new user roles to be added by subclassing without modifying existing code, thus extending functionality smoothly without causing regressions. Control-based menus within user subclasses encapsulate UI logic, allowing new menus or features to be introduced

independently, enhancing adaptability. Each class follows SRP, focusing on a single task, which further facilitates extension without affecting unrelated components. We can add new features in this code without needing to change the code much.

Maintainability is supported by low coupling and high cohesion, ensuring modules interact through well-defined interfaces and each class has clear, focused responsibilities. This minimizes side effects when making changes and makes the system easier to understand and debug. Information hiding and encapsulation safeguard internal data, reducing risks of unintended dependencies that complicate maintenance. The ECB Pattern helps isolate changes to relevant parts of the system, improving update efficiency and reducing errors.

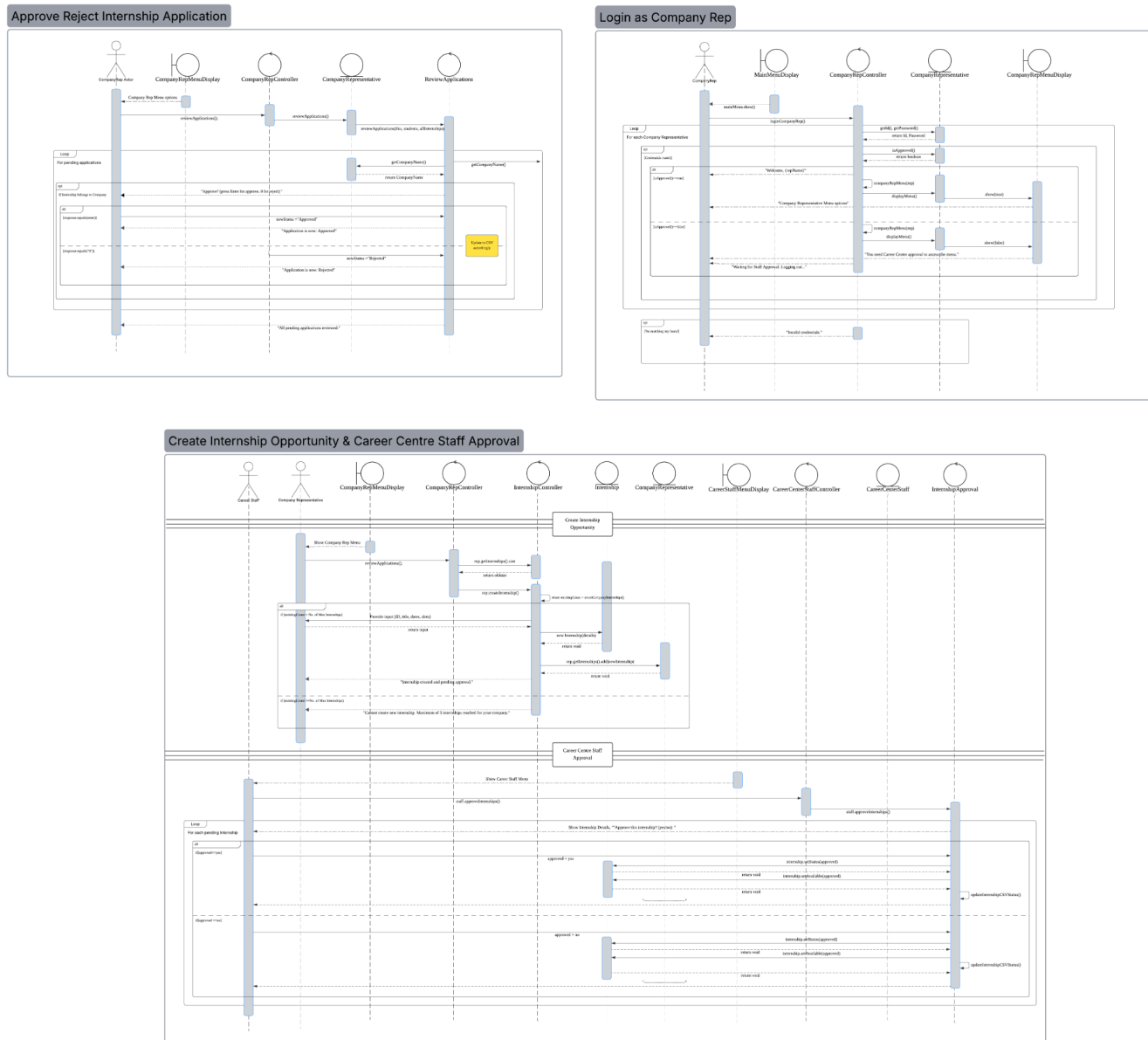
2.8. Additional Features Added:

- **Notifications:** The system notifies users when their internship application status changes from "pending" to "approved." This keeps students promptly informed about important application updates without needing to repeatedly check the system manually.
- **Bookmarks:** Students can bookmark internships they are interested in, enabling them to easily save and revisit opportunities. After bookmarking, users can view a list of all bookmarked internships along with detailed information and statuses, including whether they have applied and the current state of the application.

3. Class Diagram: (Can Refer to the clearer image uploaded on Github)



4. Sequence Diagram: (Can Refer to the clearer image uploaded on Github):



5. Testing:

Test Case 1: Main Menu Functionalities: Testing the login for Student, Company Representative, Career Center Staff and registration of new Company Representative

#	Description	Action	Expected Output	Test Result
1.	Login	User selects '1' for student, '2' for company representative, '3' for career staff and enter credentials	If credentials are correct, display welcome message and the respective menu, if incorrect display "invalid credentials" and return to main menu	Pass

4.	Register as new Company Representative	Select '4', enter his details	Registration complete. Awaiting staff approval.	Pass
6.	Login attempt by an unapproved Company Representative	Select '3', enter credentials	Display “Your account is pending approval by Career Center staff.” and return to main menu	Pass
7.	Exit	Select '5' in the menu	Displays “Goodbye” and terminate the session	Pass
8.	Invalid Menu Option	Invalid input	Invalid choice.	Pass

Test Case 2: Student Menu Functionalities: Testing the the functionalities of viewing internship, view bookmarked internship, apply internship, view application status, accepting placement, withdraw application, change password and viewing notifications

#	Description	Action	Expected Output	Test Result
1	View internships	Selects “1” in student menu	Displays all eligible internships based on student’s year, major and internship’s status, visibility, and level, with an option to bookmark.	Pass
2	View bookmarks	Select “2” in the student menu	Display all the internships the student has bookmarked	Pass
3	Apply for internship	Select “3” in the student menu and enter the internship id	Display “Application submitted. Status: Pending”. If internship id incorrect, display “Internship not found”	Pass
4	Apply for internship (Student has already filled the 3 (maximum) applications)	Select “3” in the student menu and enter the correct internship id	You have reached the maximum number of applications (3).	Pass
5	Apply for internship (Student has already	Select “3” in the student menu and	Display “You have already accepted an internship. Cannot apply for more.”	Pass

	accepted an internship)	enter the correct internship id		
6	View application status	Student select “4” in the menu	Shows all application statuses along with the full internship details, if no applications, show the error	Pass
7	Accept Internship	Student select “5” in the menu	Shows approved internship IDs and lets the user choose one; updates status and auto- withdraws others. If none exist, shows “No Internship approved.”	Pass
8.	Withdraw Application	Student select “6” in the menu	Display the success message. If he has no applications, display the error message	Pass
9.	Change Password	Student select “7” in the menu	Allows user to enter a new password, then displays “Password changed successfully.”	Pass
10.	Logout	Student select option “8” in the menu	Display “Logging out...” and return to the main menu	Pass
12.	Notification	NA	When a student logs in and has an approved application, a notification appears before the menu.	Pass
12.	Invalid Menu Option	Invalid input in the menu	Display “Invalid choice”	Pass

Test Case 3: Company Representative Menu Functionalities: Testing the the functionalities of creating, editing and deleting internship, toggle the internship visibility, view all applicants, review them, view the status of their internship application, change password and logout.

#	Description	Action	Expected Output	Test Result
1	Create Internship	Selects “1” in the menu	Prompt to enter details; if format invalid, re-input. If >5 internships created, display “Cannot create more internships”.	Pass
2	Edit Internship	Selects “2” in the menu	Enter internship ID. If approved, display ‘This internship has been approved by Career Center Staff and cannot be edited.’ Otherwise, allow edits, update details, and confirm.	Pass
3.	Delete Internship	Selects “3” in the menu	Enter internship ID. If invalid, show error and return to menu; else delete from CSV and confirm success.	Pass
4.	Toggle Visibility of Internship	Selects “4” in the menu	Prompt the representative to input an internshipID to toggle and visibility status switched and updated in CSV	Pass

#	Description	Action	Expected Output	Test Result
1	Create Internship	Selects “1” in the menu	Prompt to enter details; if format invalid, re-input. If >5 internships created, display “Cannot create more internships”.	Pass
2	Edit Internship	Selects “2” in the menu	Enter internship ID. If approved, display ‘This internship has been approved by Career Center Staff and cannot be edited.’ Otherwise, allow edits, update details, and confirm.	Pass
5.	View all applicants	Selects “5” in the menu	Display the list of student applicants for the each internship, if no applicant, display the message	Pass
6.	Review applicants	Select “6” in the menu	Press Enter to approve or 0 to reject each applicant; show approval/rejection. If none, display ‘All Pending Applications Reviewed.	Pass
7.	View Internship approval status	Selects “7” in the menu	Shows all internship application statuses along with, if no applications, show the error	Pass
9.	Change Password	Selects “8” in the menu	Allows user to enter a new password, then displays “Password changed successfully.”	Pass
10.	Logout	Selects “9” in the menu	Display “Logging out...” and return to the main menu	Pass
12.	Invalid Menu Option	Invalid input in the menu	Display “Invalid choice”	Pass

Test Case 4: Career Center Staff Menu Functionalities: Testing the the functionalities of approving company representatives, approve internships, handle withdrawal requests, generating reports, change password and logout.

#	Description	Action	Expected Output	Test Result
1	Approve Company Representative	Enters “1” in the menu	Prompt yes/no for each career rep; show approval/rejection message accordingly. If none, display ‘No pending company representatives to approve.	Pass
2	Approve internship	Enters “2” in the menu	Prompt yes/no for each internship; show approval/rejection. If none, display ‘No pending internships to approve.’	Pass

3	Withdrawal requests	Enters “3” in the menu	Prompt yes/no for each withdrawal request; show approval/rejection. If none, display ‘No withdrawal requests.’	Pass
4.	Generating reports	Enters “4” in the menu	System displays the data of all internships	Pass
5.	Change Password	Select “5” in the menu	Allows user to enter a new password, then displays “Password changed successfully.”	Pass
6.	Logout	Select “5” in the menu	Display “Logging out...” and return to the main menu	Pass
7.	Invalid Menu Option	Invalid input in the menu	Display “Invalid choice”	Pass

6. Reflection

As a group, we faced quite a few challenges during this assignment, particularly in balancing different ideas and approaches while still working towards a unified design and ensuring our individual components integrated smoothly. Early on, we struggled to align our understanding of the system requirements and maintain consistent coding practices across the team. Through regular discussions, clearer task delegation, and iterative reviews, we managed to overcome these issues and improve our workflow. We often found ourselves revisiting earlier decisions because implementing one part of the system would reveal gaps or inefficiencies in another. On top of that, not having a centralized workspace at the start created a lot of confusion as some of us were working on outdated versions while others were making changes in separate files. This led to overlapping work and unnecessary re-implementation, which only slowed us down. To address this, we gradually learned to divide the workload into clearer stages. First we discussed the design together, finalized the data structures and only then divided the implementation into smaller, well-defined tasks for proper execution and delegation. This approach helped reduce confusion, prevent overlapping work, and ensure that our components stayed compatible. This project strengthened our understanding of good design principles, particularly encapsulation, inheritance, polymorphism, and abstraction. We saw how encapsulation helps keep data protected and organised within each class, how inheritance allows us to reuse structures efficiently, how polymorphism lets different components interact through common interfaces, and how abstraction simplifies complex behaviour into clearer, more manageable designs. Applying these concepts and principles in practice helped us see how they directly contribute to cleaner architecture, easier debugging and maintenance, and smoother integration across different compartments which would greatly boost usability. Additionally, implementing role-based access control and automated notifications would enhance security and streamline workflows.